

REPUBLIQUE DEMOCRATIQUE DU CONGO
UNIVERSITE PEDAGOGIQUE NATIONALE
FACULTE DE SCIENCE ET TECHNOLOGIE
DEPARTEMENT DE MATH, STAT ET INFO
OPTION : GENIE LOGICIEL
MASTER 1



BP.8815

CONCEPTION D'UN NAVIGATEUR WEB PERMETTANT DE
COLLECTER DES INFORMATIONS PERTINENTES DES
UTILISATEURS

Fait par :

MULAMBA MUKOMA Olivier
NTUMBA MULUMBA Belge
MAKENGO KIESE Exaucece
DENIS MASUDI Daniel
KWETO Merveille

Professeur
Mayala Lemba Francis

Kinshasa
Année Académique 2025-2025

CONCEPTION D'UN NAVIGATEUR WEB PERMETTANT DE COLLECTER DES INFORMATIONS PERTINENTES SUR LES ACTIVITÉS DE NAVIGATION DES UTILISATEURS

I. DÉFINITIONS IMPORTANTES DANS LE CADRE DE NOTRE TRAVAIL

Navigateur Web : un navigateur Web est un logiciel qui permet aux utilisateurs d'accéder aux ressources disponibles sur Internet, notamment les pages Web, les images, les documents et les services en ligne. Dans le cadre de notre travail, nous avons développé un navigateur de bureau appelé *Odem Browser*, qui permet de consulter des pages Web et de transmettre certaines informations relatives à la navigation à une API REST.

Serveur Web : un serveur Web est un logiciel qui reçoit les requêtes envoyées par les applications clientes et leur fournit les ressources ou les informations demandées. Il joue un rôle essentiel dans les échanges entre les applications connectées à un réseau.

Requête HTTP : une requête HTTP est un message envoyé par une application cliente à un serveur pour demander une ressource ou effectuer une opération. Dans notre projet, ce mécanisme permet notamment au navigateur de transmettre des informations à l'API REST et de récupérer les données enregistrées.

API REST : une API (*Application Programming Interface*) est une interface qui permet à plusieurs applications de communiquer entre elles. Une API REST utilise les mécanismes du protocole HTTP pour exposer des ressources et des fonctionnalités. Dans notre solution, elle assure la réception, le traitement et la consultation des données transmises par le navigateur.

Base de données : une base de données est un système permettant de stocker et d'organiser des informations afin de faciliter leur consultation et leur mise à jour. Dans notre projet, elle sert à conserver les informations reçues par l'API REST.

Architecture logicielle : l'architecture logicielle désigne l'organisation des différents composants d'une application et la manière dont ils communiquent entre eux. Elle permet de répartir les responsabilités et de faciliter le développement, la maintenance et l'évolution du système.

Collecte des données : la collecte des données consiste à recueillir des informations répondant à un objectif précis. Dans notre travail, elle concerne les informations nécessaires à l'étude des activités de navigation, notamment les URL visitées et les dates et heures correspondantes.

II. ARCHITECTURE DE NOTRE SOLUTION

II.1. Présentation des composants de la solution

Notre solution repose sur une architecture client-serveur composée de trois éléments principaux : le navigateur Web, l'API REST et la base de données. Une interface d'administration peut également être utilisée pour faciliter la consultation des informations collectées.

Le navigateur Web

Le navigateur constitue la partie visible de notre application. Développé en Python avec PyQt5 et Qt WebEngine, il permet à l'utilisateur d'accéder aux pages Web et d'effectuer les opérations habituelles de navigation.

En plus de ces fonctionnalités, le navigateur intègre un mécanisme de communication avec l'API REST. Celui-ci permet de transmettre certaines informations relatives aux activités de navigation afin qu'elles puissent être enregistrées et consultées ultérieurement.

L'API REST

L'API REST assure la communication entre le navigateur et la base de données. Développée avec FastAPI, elle reçoit les informations transmises par le navigateur, les traite et les enregistre.

Elle expose également une route permettant de récupérer les données enregistrées. Cette organisation facilite les échanges entre les applications et permet de séparer la logique de navigation du traitement des données.

La base de données

La base de données permet de conserver les informations reçues par l'API. Notre projet utilise SQLAlchemy pour faciliter les interactions entre le code Python et le système de stockage.

Les informations enregistrées peuvent ainsi être consultées ultérieurement sans qu'il soit nécessaire de les recueillir à nouveau auprès du navigateur.

L'interface d'administration

L'interface d'administration a pour objectif de faciliter la consultation des données collectées. Elle peut notamment présenter les URL enregistrées, les adresses IP et les dates et heures correspondantes.

Elle communique avec l'API REST pour récupérer les informations et les présenter de manière organisée. Son implémentation doit toutefois être distinguée de celle du navigateur et de l'API.

II.2. Fonctionnement général de la solution

Le fonctionnement de notre solution repose sur un échange d'informations entre le navigateur, l'API REST et la base de données.

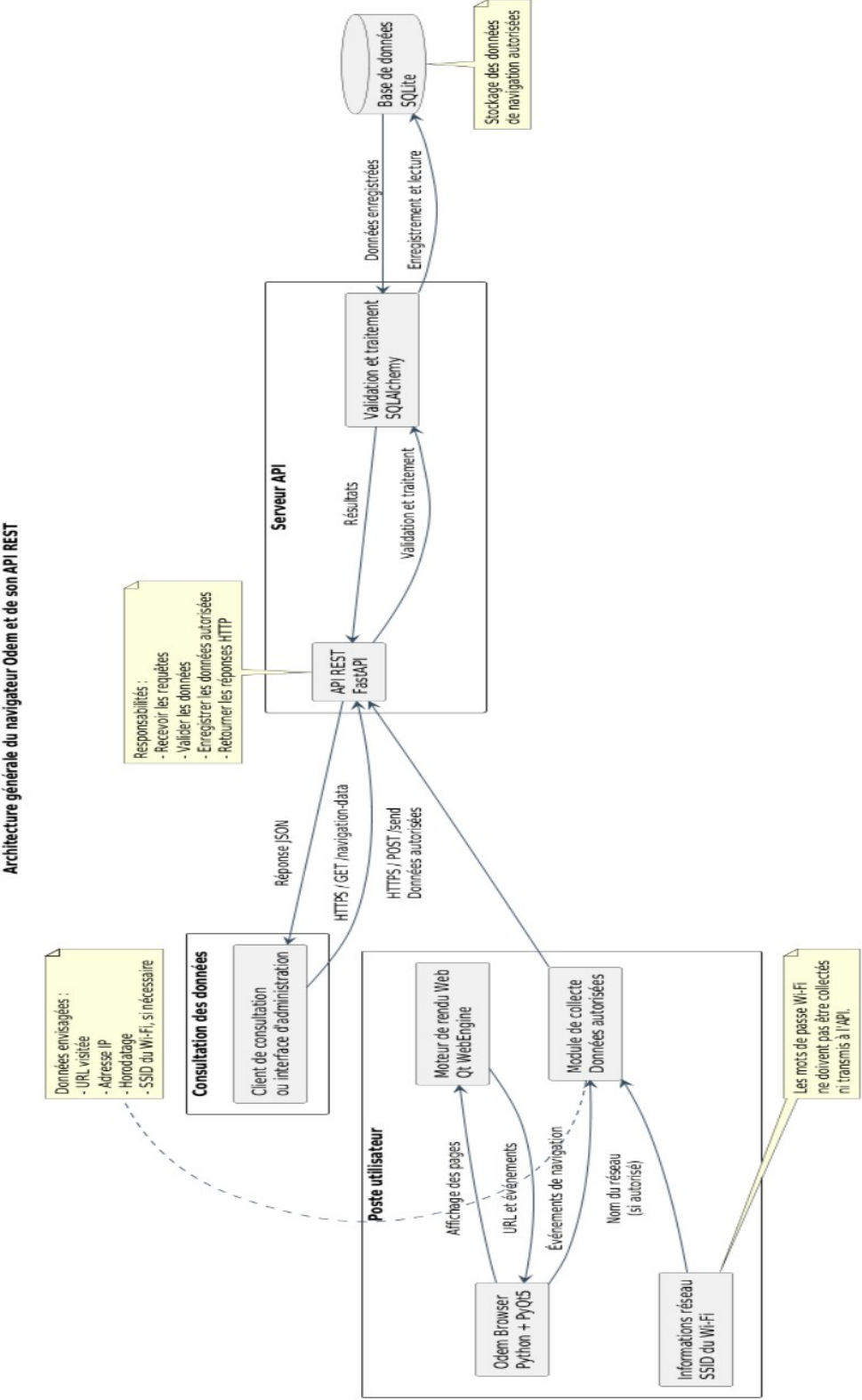
Tout commence lorsque l'utilisateur saisit une adresse Web dans le navigateur. Celui-ci charge la page demandée et prépare les informations nécessaires à la collecte.

Les informations prévues par le système sont ensuite transmises à l'API REST au moyen d'une requête HTTP. À la réception de cette requête, l'API traite les données et procède à leur enregistrement dans la base de données.

Une fois les informations enregistrées, elles peuvent être récupérées à l'aide d'une requête de consultation. Une interface d'administration peut alors les afficher sous forme de tableau ou les exploiter pour produire des statistiques.

Ce fonctionnement permet de distinguer trois opérations essentielles : la collecte des informations par le navigateur, leur traitement et leur stockage par l'API, puis leur consultation par une application autorisée.

Figure 1 : Architecture générale de notre solution.



III. IMPLÉMENTATION DE LA SOLUTION

III.1. Développement du navigateur Web

Nous avons développé notre navigateur Web avec le langage Python, en utilisant la bibliothèque graphique PyQt5 et le composant Qt WebEngine.

Le choix de Python s'explique notamment par sa simplicité et par la disponibilité de bibliothèques permettant de développer rapidement des applications de bureau. PyQt5 facilite la création de l'interface graphique, tandis que Qt WebEngine permet d'afficher les pages Web directement dans l'application.

Le navigateur, nommé *Odem Browser*, dispose d'une barre d'adresse et de plusieurs commandes permettant de naviguer sur Internet. L'utilisateur peut ouvrir une page, revenir à la page précédente, avancer vers la page suivante et actualiser le contenu affiché.

L'application propose également une page d'accueil personnalisée et un accès à certains services en ligne, notamment WhatsApp Web.

Au-delà de ces fonctionnalités classiques, nous avons intégré un mécanisme de communication avec notre API REST. Ce mécanisme permet au navigateur de transmettre les informations nécessaires à la collecte lorsqu'un événement de navigation est détecté.

L'objectif est de réunir, au sein d'une même application, les fonctionnalités essentielles d'un navigateur Web et un mécanisme de transmission des données vers un serveur distant.

III.2. Réalisation de l'interface graphique avec PyQt5

L'interface graphique représente le principal moyen d'interaction entre l'utilisateur et le navigateur. Nous l'avons développée avec PyQt5 afin de proposer un environnement simple et facile à utiliser.

Elle comprend notamment une barre d'adresse, une zone d'affichage des pages Web et plusieurs boutons permettant de contrôler la navigation.

La barre d'adresse permet de saisir l'URL de la page à consulter. Les boutons de navigation permettent de revenir en arrière, d'avancer ou d'actualiser la page courante.

La zone centrale de l'interface est assurée par Qt WebEngine, qui prend en charge le chargement et l'affichage du contenu Web.

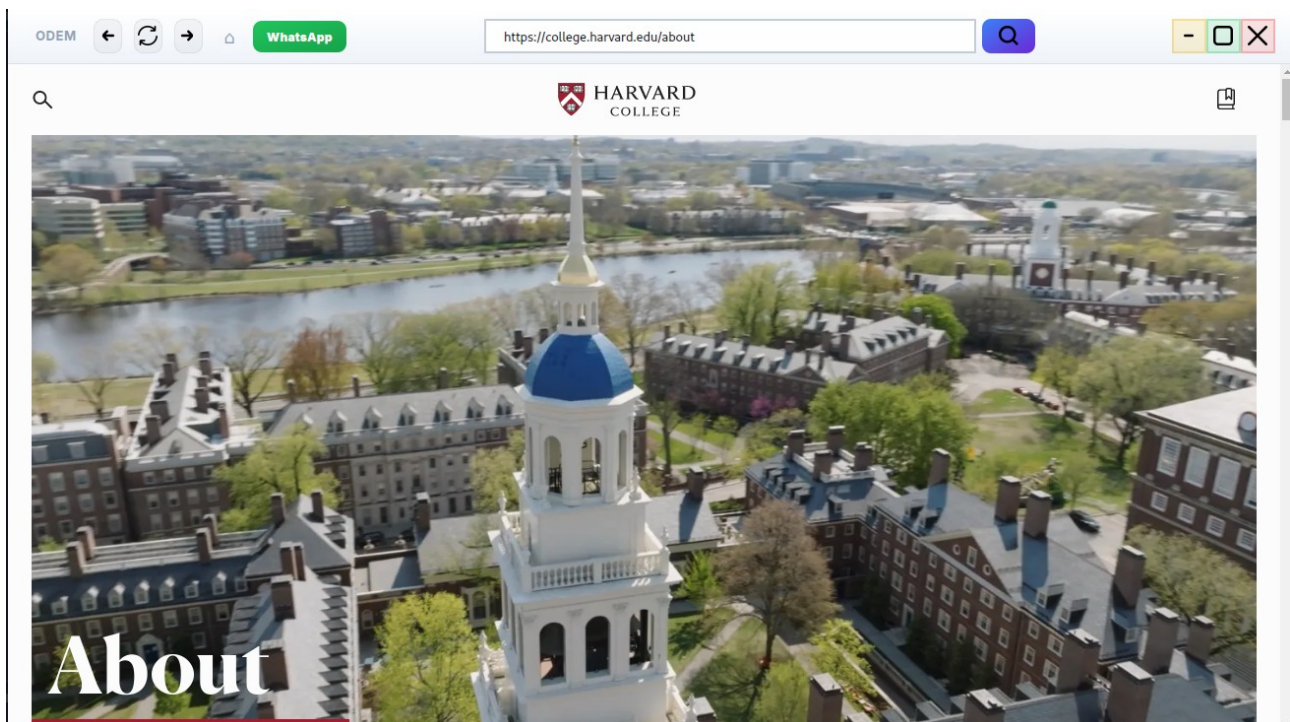
Nous avons également organisé le code de manière à relier les actions de l'utilisateur aux fonctions correspondantes. Ainsi, lorsqu'un bouton est utilisé, l'application exécute l'opération prévue sans que l'utilisateur ait besoin d'intervenir directement sur le code.

Cette organisation contribue à rendre l'application plus intuitive et facilite les évolutions futures de son interface.

Figure 2 : Interface principale du navigateur Odem Browser.



Figure 3 : Exemple de recherche effectuée dans le navigateur.



III.3. Développement de l'API REST

Pour assurer la communication entre le navigateur et la base de données, nous avons développé une API REST avec FastAPI.

Ce framework Python nous permet de définir des routes HTTP et de traiter les données échangées entre les applications. Il facilite également l'organisation du code et la validation des informations reçues.

Dans notre projet, l'API joue un rôle intermédiaire entre le navigateur et le système de stockage. Elle reçoit les données transmises, les traite et les enregistre avant de les rendre accessibles aux applications autorisées.

Deux routes principales sont prévues dans l'API.

La route POST /send

Cette route permet au navigateur de transmettre les informations relatives à un événement de navigation. Les données sont envoyées au format JSON dans une requête HTTP.

Après réception, l'API traite les informations et les enregistre dans la base de données. Cette opération permet de conserver une trace des événements transmis par le navigateur.

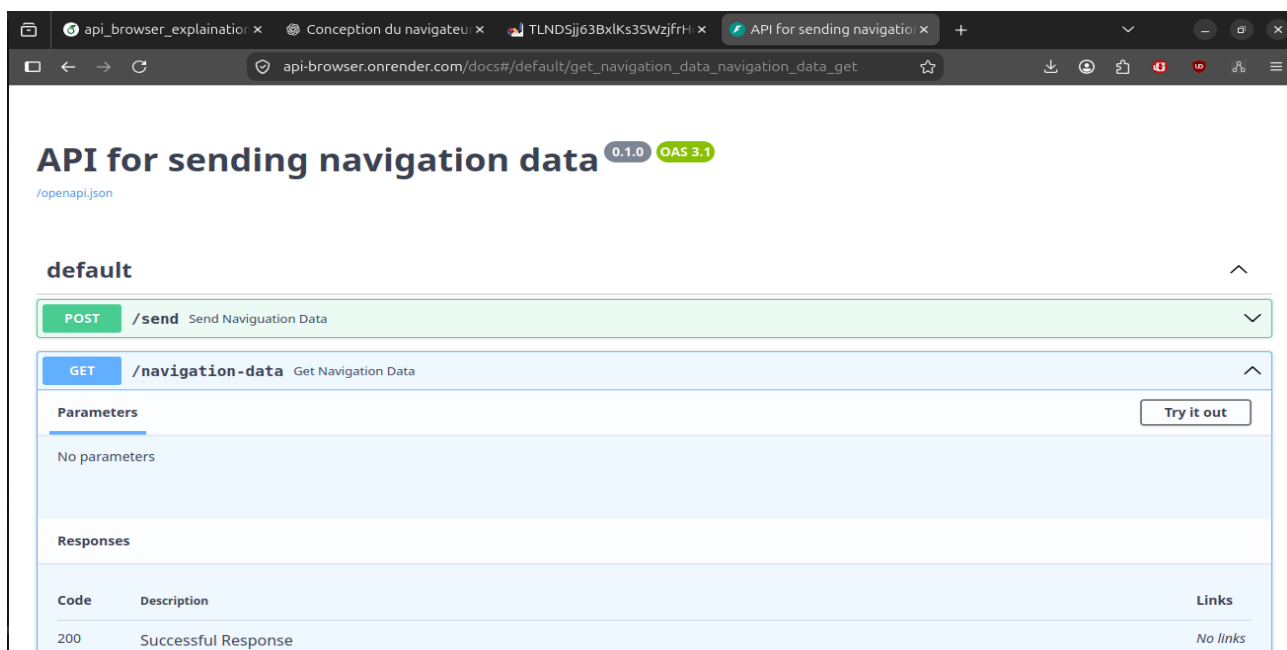
La route GET /navigation-data

Cette route permet de consulter les données enregistrées. Lorsqu'une application envoie une requête à cette adresse, l'API récupère les enregistrements disponibles et les retourne sous forme de réponse JSON.

Cette fonctionnalité facilite la consultation des données et permet à d'autres applications de les exploiter sans accéder directement à la base de données.

L'utilisation de ces deux routes permet ainsi de séparer l'opération d'enregistrement de celle de consultation.

Figure 4 : Documentation de l'API REST avec FastAPI.



III.4. Stockage des données avec SQLAlchemy et SQLite3

La conservation des informations collectées constitue une étape importante de notre système. Pour cela, nous utilisons SQLAlchemy afin de faciliter les opérations effectuées sur la base de données.

SQLAlchemy est un ORM qui permet de manipuler les données à travers des objets Python plutôt que de construire manuellement chaque opération SQL. Il facilite notamment la création des enregistrements, leur consultation et la gestion des sessions de base de données.

Le projet comprend également une configuration de connexion et des modèles représentant les données utilisées par l'application.

Les principales informations de navigation prévues comprennent l'adresse IP, l'adresse MAC transmise par le client, l'URL, le mot de passe et le nom du wifi et l'horodatage.

Chaque information possède un rôle particulier. L'URL permet d'identifier la ressource associée à l'événement, tandis que l'horodatage permet de situer cet événement dans le temps. Les informations réseau peuvent apporter un contexte supplémentaire, mais elles ne permettent pas, à elles seules, d'établir avec certitude l'identité d'un utilisateur.

SQLite constitue une solution adaptée à un prototype, car elle permet de stocker les données dans un fichier sans nécessiter un serveur de base de données indépendant.

L'association de SQLAlchemy et de SQLite3 facilite donc la mise en œuvre du système et permet de conserver les données collectées pour une consultation ultérieure.

Organisation des données enregistrées dans la base de données

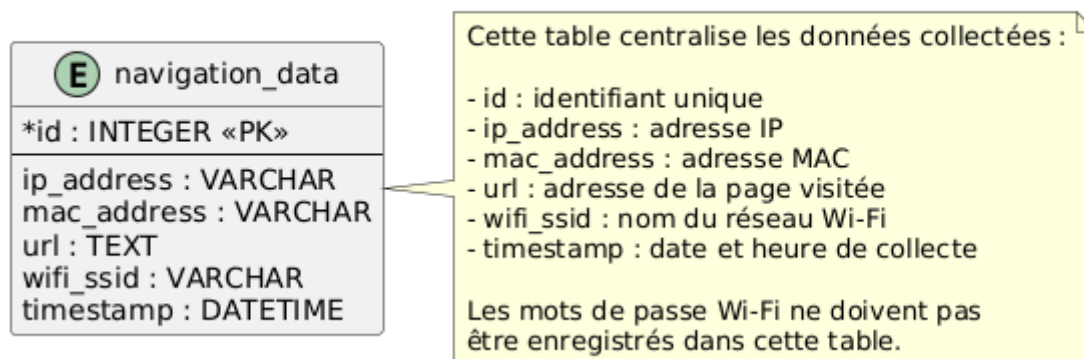


Figure 5 : Organisation des données enregistrées dans la base de données.

III.5. Consultation et exploitation des données

Les informations enregistrées par l'API peuvent être exploitées afin de mieux comprendre les événements de navigation pris en charge par le système.

Une interface de consultation peut présenter les données dans un tableau comprenant l'URL, l'adresse IP, l'adresse MAC transmise et la date de l'événement.

Une telle présentation facilite la lecture des informations et permet de retrouver plus facilement les enregistrements recherchés.

Il est également possible d'envisager l'ajout de fonctions de recherche et de filtrage, par exemple pour consulter les événements enregistrés pendant une période déterminée.

Par ailleurs, les données peuvent servir à produire des statistiques descriptives, telles que le nombre d'événements enregistrés par jour ou la répartition des URL observées.

Ces possibilités montrent que l'API ne se limite pas à recevoir des informations : elle fournit également une base technique pour leur consultation et leur exploitation.

Toutefois, les fonctions de visualisation et d'administration doivent être présentées comme des fonctionnalités opérationnelles uniquement si leur implémentation a été réalisée et testée.

IV. TESTS ET VALIDATION

Afin de vérifier le bon fonctionnement de notre solution, plusieurs catégories de tests peuvent être réalisées. Elles concernent le navigateur, l'API REST et les échanges entre les différents composants.

IV.1. Tests du navigateur Web

Les tests du navigateur permettent de vérifier que les fonctionnalités principales sont accessibles et fonctionnent comme prévu.

Fonctionnalité	Test réalisé	Résultat attendu
Démarrage	Lancer l'application.	La fenêtre principale s'affiche.
Chargement d'une page	Saisir une URL valide.	La page demandée s'affiche.
Retour en arrière	Cliquer sur le bouton précédent.	La page précédente s'affiche.
Navigation avant	Cliquer sur le bouton suivant.	La page suivante s'affiche.
Actualisation	Cliquer sur le bouton d'actualisation.	La page est rechargée.
Mise à jour de l'adresse	Charger une nouvelle page.	La barre d'adresse affiche l'URL courante.
Fermeture	Fermer la fenêtre.	L'application se ferme normalement.

Les résultats de ces tests doivent être consignés après leur exécution afin de déterminer si chaque fonctionnalité répond aux attentes.

IV.2. Tests de l'API REST

Les tests de l'API visent à vérifier que les données peuvent être reçues, enregistrées et consultées correctement.

Test	Opération	Résultat attendu
Transmission des données	Envoyer une requête POST /send avec des données valides.	Les données sont enregistrées.
Validation des données	Envoyer des données non conformes au schéma.	L'API retourne une erreur de validation.
Consultation des données	Envoyer une requête GET /navigation-data.	Les données enregistrées sont retournées.
Base de données vide	Consulter les données lorsqu'aucun enregistrement n'existe.	L'API retourne une liste vide.

Test	Opération	Résultat attendu
Indisponibilité du serveur	Effectuer une requête lorsque l'API est inaccessible.	Le client gère l'échec de communication.

Ces essais permettent de vérifier le comportement des principales routes et de détecter d'éventuelles erreurs dans le traitement des données.

IV.3. Tests d'intégration

Les tests d'intégration permettent de vérifier le fonctionnement de l'ensemble du système.

Pour cela, nous pouvons ouvrir le navigateur, charger une page Web, puis vérifier si les informations correspondantes sont transmises à l'API et enregistrées dans la base de données.

La consultation de la route GET /navigation-data permet ensuite de vérifier si l'enregistrement est effectivement disponible.

Cette démarche permet d'évaluer le fonctionnement de la chaîne complète, depuis l'événement de navigation jusqu'à la conservation des informations.

Les résultats obtenus doivent être documentés afin de confirmer le bon fonctionnement du système ou d'identifier les corrections nécessaires.

V. LIMITES ET PERSPECTIVES

V.1. Limites de la solution

Malgré les fonctionnalités développées, notre solution présente certaines limites.

Tout d'abord, la qualité des données dépend de la manière dont les événements de navigation sont détectés. Un changement d'URL ne correspond pas nécessairement à une visite distincte, ce qui peut entraîner des doublons.

Ensuite, les informations réseau ne permettent pas toujours d'identifier de manière fiable un utilisateur. Les adresses IP peuvent être partagées et les adresses MAC peuvent être indisponibles ou modifiées.

Par ailleurs, le système nécessite des mécanismes de sécurité adaptés pour empêcher les accès non autorisés aux données.

Enfin, les possibilités d'analyse restent limitées tant que des fonctions supplémentaires de filtrage, de visualisation et de production de statistiques ne sont pas intégrées et testées.

V.2. Perspectives d'amélioration

Plusieurs améliorations peuvent être envisagées pour faire évoluer notre solution.

Une première perspective consiste à améliorer la détection des événements de navigation afin de réduire les doublons et d'assurer une meilleure cohérence des données.

Une deuxième amélioration concerne la sécurité. L'ajout de mécanismes d'authentification, de contrôle d'accès et de protection des données permettrait de renforcer la fiabilité du système.

Il serait également intéressant de développer une interface d'administration permettant de rechercher, de filtrer et de visualiser les informations enregistrées.

L'ajout de graphiques statistiques pourrait faciliter l'analyse des données et permettre de mieux comprendre leur évolution dans le temps.

Enfin, l'automatisation des tests contribuerait à vérifier plus facilement le fonctionnement des composants et à détecter les régressions lors des mises à jour.

CONCLUSION

Ce travail nous a permis d'aborder la conception d'un navigateur Web capable de communiquer avec une API REST afin de transmettre et de conserver des informations relatives aux activités de navigation.

Le navigateur Odem Browser a été développé avec Python, PyQt5 et Qt WebEngine. Il permet à l'utilisateur de consulter des pages Web tout en intégrant un mécanisme de communication avec le serveur.

L'API REST, développée avec FastAPI, assure la réception et la consultation des données. SQLAlchemy facilite les interactions avec la base de données, dans laquelle les informations peuvent être conservées pour une exploitation ultérieure.

L'architecture adoptée permet de séparer les responsabilités des différents composants et facilite leur évolution. Elle constitue également une base pour le développement de fonctionnalités complémentaires, notamment la consultation des données et leur analyse statistique.

Des tests fonctionnels et des tests d'intégration sont nécessaires pour confirmer le bon fonctionnement de l'ensemble de la solution. Enfin, la collecte des données doit être réalisée de manière transparente, sécurisée et limitée aux informations nécessaires.

Ce projet constitue ainsi une base de travail pour l'étude des échanges entre un navigateur Web, une API REST et une base de données, tout en ouvrant des perspectives d'amélioration en matière de sécurité, de visualisation et d'exploitation des informations collectées.